

Policy for explicit

Document #: P3116R0
Date: 2024-02-05
Project: Programming Language C++
Audience: LEWG
LWG
Reply-to: Zach Laine
<whatwasthataddress@gmail.com>

Contents

- 1 Changelog
- 2 Motivation
- 3 Requirements for this policy
 - 3.1 (1) A rationale, and (5) A rationale as to why adopting the policy will improve coherence and / or save time.
 - 3.2 (2) A survey of the prior art of the topic, as appears in the standard.
 - 3.2.1 [support] (9 occurrences)
 - 3.2.2 [diagnostics] (26 occurrences)
 - 3.2.3 [mem] (25 occurrences)
 - 3.2.4 [utilities] (83 occurrences)
 - 3.2.5 [strings] (8 occurrences)
 - 3.2.6 [containers] (87 occurrences)
 - 3.2.7 [iterators] (10 occurrences)
 - 3.2.8 [ranges] (138 occurrences)
 - 3.2.9 [numerics] (90 occurrences)
 - 3.2.10 [time] (47 occurrences)
 - 3.2.11 [localization] (38 occurrences)
 - 3.2.12 [input.output] (110 occurrences)
 - 3.2.13 [re] (7 occurrences)
 - 3.2.14 [thread] (43 occurrences)

- 3.3 (3) A survey of the status quo for this topic, in the wider C++ community.
- 4 Proposal
 - 4.1 (4) A clear and concise definition of the policy proposed.
 - 4.2 (6) Proposed changes to the wording of SD-9.
- 5 References

§ 1 Changelog

Initial revision.

§ 2 Motivation

Based on the recently-adopted P2267R1: Library Evolution Policies, this paper makes the case for a policy for when and why to add `explicit` to member functions in standard library templates and types.

There are 721 occurrences of the keyword `explicit` in the library clauses of the standard. This does not include the appearance of the keyword in plain text; this count is of in-code appearances only. This is an overcount, though, as I overcount duplicates when an `explicit` member function is listed in a synopsis and then again in a definition. However, this gives the sense of how many times `explicit` is currently used.

These occurrences fall into three distinct categories:

1. `explicit` operator `bool()`
2. `explicit` on single-parameter constructors to prevent undesirable implicit conversions.
3. `explicit` on zero-parameter constructors for tag types, to enforce spelling the tag type's name at the point of construction.

In hundreds of uses of `explicit`, I found almost no uses of `explicit` that did not fall under one of the three categories above (except for views in [ranges] section, discussed below). This is a pretty good indication that we already use `explicit` consistently, and that formalizing this consistency would make a good policy.

§ 3 Requirements for this policy

P2267R1: Library Evolution Policies defines requirements for a new policy. Below is a discussion of the required elements, based on section 5.1.1 of that paper, using the same numbering from that paper.

§ 3.1 (1) A rationale, and (5) A rationale as to why adopting the policy will improve coherence and / or save time.

We have consistent existing practice for how `explicit` is used in the standard library. We also sometimes have to remind authors during design review to use `explicit`. Having a policy would save LEWG effort by concretely indicating to paper authors when and where `explicit` should be used. A policy will also help paper authors keep uses of `explicit` to the kinds of uses indicated in the policy, rather than other ad hoc uses; this will further help improve coherence.

This rationale applies only to turning the three categories of use above into policy. There are some uses of `explicit` in the standard that are exceptions to those categories.

- The use of `explicit` on deduction guides (only three in the entire standard). This appears to be addressed by LWG3451, and so this exception can be ignored.
- The use of `explicit` with multi-parameter view constructors. This seems to be a special case; this one of only two places I know about `explicit` is used on multi-parameter constructors (the other is next in this list). This does not seem like something that we should encourage in future proposals, for the reasons outlined below, in the section on the `[ranges]` clause.
- The conditional use of `explicit` with multi-parameter span constructors. As described in [P1976R2], the conditional use of `explicit` is intended to make it less likely for the user to write `UB` into their code when constructing a `non_dynamic_extent` span from a pointer and size or a pair of iterators. This also seems like a fairly particular use of `explicit`, ill-suited to be turned into a broad rule.
- The use of `explicit` on select conversion operators in classes in `[time]`. The usage there is to make an explicit conversion possible with a cast, while disallowing that conversion implicitly. This is certain to come up from time to time, but not often. For now, it seems best not to try to make this use of `explicit` part of the policy. We can always add it later if it becomes common.

The take-away here is that the three rules proposed in this paper are applied with a high level of consistency in the standard today. Though there are plenty of other uses of `explicit`, as outlined above, none of those uses is contrary to the three rules proposed.

§ 3.2 (2) A survey of the prior art of the topic, as appears in the standard.

Here is a breakdown of the use of `explicit` in the library clauses, clause by clause.

§ 3.2.1 [support] (9 occurrences)

Used on `partial/strong/weak_ordering`, e.g.:

```
constexpr explicit partial_ordering(ord v) noexcept;
```

explicit is used to prevent implicit conversions from (exposition-only enum) ord to partial_ordering.

§ 3.2.2 [diagnostics] (26 occurrences)

Used on the constructors for various exception types that take a string, and the constructors of various types that take an allocator, e.g.:

```
explicit logic_error(const string& what_arg);  
explicit basic_stacktrace(const allocator_type& alloc) noexcept;
```

Used in explicit operator bool() const noexcept.

§ 3.2.3 [mem] (25 occurrences)

Used on various smart pointers' constructors, e.g.:

```
template<class U> constexpr explicit unique_ptr(U p) noexcept;  
template<class Y> explicit shared_ptr(Y* p);
```

Used in explicit operator bool() const noexcept.

§ 3.2.4 [utilities] (83 occurrences)

Used on the constructors of various utility templates, e.g.:

```
template<class... Args>  
    constexpr explicit optional(in_place_t, Args&&...);  
template<size_t I, class... Args>  
    constexpr explicit variant(in_place_index_t<I>, Args&&...);  
template<class T, class... Args>  
    explicit any(in_place_type_t<T>, Args&&...);  
template<class Err = E>  
    constexpr explicit unexpected(Err&&);  
template<class... Args>  
    constexpr explicit expected(in_place_t, Args&&...);  
template<class charT, class traits, class Allocator>  
    constexpr explicit bitset(  
        const basic_string<charT, traits, Allocator>& str,  
        typename basic_string<charT, traits, Allocator>::size_type pos = 0,  
        typename basic_string<charT, traits, Allocator>::size_type n  
            = basic_string<charT, traits, Allocator>::npos,  
        charT zero = charT('0'),  
        charT one = charT('1'));  
template<class T, class... Args>  
    explicit move_only_function(in_place_type_t<T>, Args&&...);  
template<class T, class... Args>  
    explicit copyable_function(in_place_type_t<T>, Args&&...);
```

```
constexpr explicit basic_format_parse_context(basic_string_view<charT>
    fmt) noexcept;
template<class T> explicit basic_format_arg(T& v) noexcept;
```

Used on various exceptions' constructors.

Used in explicit operator bool() const noexcept.

§ 3.2.5 [strings] (8 occurrences)

Used on basic_string and basic_string_view constructors, e.g.:

```
template<class R>
    constexpr explicit basic_string_view(R&& r);

constexpr explicit basic_string(const Allocator& a) noexcept;
```

Used on two basic_string deduction guides:

```
template<class charT,
    class traits,
    class Allocator = allocator<charT>>
    explicit basic_string(basic_string_view<charT, traits>, const Allocator&
        = Allocator())
        -> basic_string<charT, traits, Allocator>;

template<class charT,
    class traits,
    class Allocator = allocator<charT>>
    basic_string(basic_string_view<charT, traits>,
        typename see below::size_type, typename see
        below::size_type,
        const Allocator& = Allocator())
        -> basic_string<charT, traits, Allocator>;
```

These guides are part of the subject of LWG3451 (see below).

§ 3.2.6 [containers] (87 occurrences)

Used on constructors that may take a single argument, e.g.:

```
explicit deque(const Allocator&);

explicit unordered_map(size_type n,
    const hasher& hf = hasher(),
    const key_equal& eql = key_equal(),
    const allocator_type& a = allocator_type());

explicit queue(const Container& cont);

template<class... OtherIndexTypes>
    constexpr explicit extents(OtherIndexTypes...) noexcept;
```

```
template<class... OtherIndexTypes>
    constexpr explicit mdspan(data_handle_type ptr, OtherIndexTypes...
        exts);
```

Used conditionally on various span multi-argument constructors:

```
template<class It>
    constexpr explicit(extent != dynamic_extent) span(It first, size_type
        count);
template<class It, class End>
    constexpr explicit(extent != dynamic_extent) span(It first, End last);
    constexpr explicit(extent != dynamic_extent) span(R&& r);
constexpr
    explicit(extent != dynamic_extent)
        span(std::initializer_list<value_type> il);
template<class OtherElementType, size_t OtherExtent>
    constexpr explicit(see below) span(const span<OtherElementType,
        OtherExtent>& s) noexcept;
```

Used in tag types like `sorted_t`.

Used in explicit operator `bool()` `const noexcept`.

§ 3.2.7 [iterators] (10 occurrences)

Used on various adapting iterators' constructors, e.g.:

```
constexpr explicit reverse_iterator(Iterator x);

constexpr explicit back_insert_iterator(Container& x);
```

§ 3.2.8 [ranges] (138 occurrences)

Used on various views' constructors that may take a single argument, e.g.:

```
constexpr explicit single_view(const T& t) requires copy_constructible<T>;

constexpr explicit iota_view(W value);
```

Used on various views' nested iterator and adaptor types' constructors, e.g.:

```
// Within struct iota_view<W, Bound>::iterator.
constexpr explicit iterator(W value);
```

Used on the `join_view` deduction guide:

```
template<class R>
    explicit join_view(R&&) -> join_view<views::all_t<R>>;
```

This is the subject of LWG3451, “Inconsistently explicit deduction guides” (<https://www.open-std.org/jtc1/sc22/wg21/docs/lwg-active.html#3451>), along with the `basic_string` deduction guides mentioned previously. The issue recommends that the `explicit` be struck from all three deduction guides. The issue seems to have been accepted as P3; these three deduction guides seem to be safe to ignore.

Used in explicit operator bool() const noexcept.

There are a few templates in this clause that have intentionally non-explicit single-argument constructors:

```
template<class R, class Allocator = allocator<byte>>
    elements_of(R&&, Allocator = Allocator()) -> elements_of<R&&,
        Allocator>;

template<different-from<ref_view> T>
    requires see below
constexpr ref_view(T&& t);

constexpr owning_view(R&& t);
```

There are also a few view templates in this clause that have multi-argument constructors that are marked explicit:

```
constexpr explicit iota_view(type_identity_t<W> value,
    type_identity_t<Bound> bound);
constexpr explicit iota_view(iterator first, see below last);
constexpr explicit repeat_view(piecewise_construct_t,
    tuple<TArgs...> value_args,
    tuple<BoundArgs...> bound_args = tuple<>{});
constexpr explicit repeat_view(piecewise_construct_t,
    tuple<TArgs...> value_args,
    tuple<BoundArgs...> bound_args = tuple<>{});
constexpr explicit transform_view(V base, F fun);
constexpr explicit take_view(V base, range_difference_t<V> count);
constexpr explicit take_while_view(V base, Pred pred);
constexpr explicit drop_view(V base, range_difference_t<V> count);
constexpr explicit drop_while_view(V base, Pred pred);
constexpr explicit join_with_view(V base, Pattern pattern);
constexpr explicit lazy_split_view(R&& r, range_value_t<R> e);
constexpr explicit split_view(V base, Pattern pattern);
constexpr explicit adjacent_transform_view(V base, F fun);
constexpr explicit chunk_view(V base, range_difference_t<V> n);
constexpr explicit slide_view(V base, range_difference_t<V> n);
constexpr explicit chunk_by_view(V base, Pred pred);
constexpr explicit stride_view(V base, range_difference_t<V> stride);
```

This differs from other templates in the standard. There is an associated paper [P2711R1]: Making multi-param constructors of views explicit, which was discussed at the Kona 2022 meeting.

The intention here is to eliminate the difference in user code between a single- and multi-argument call to one of these view's constructors. If they're all explicit, then `foo_view x = {arg};` and `foo_view x = {arg1, arg2};` will both be ill-formed, which is more consistent. As the paper points out, users will almost always be using the range adaptors and not constructing views directly, and the inconsistency problem itself it not very serious.

§ 3.2.9 [numerics] (90 occurrences)

Used on various templates in random number generation, e.g.:

```
explicit linear_congruential_engine(result_type s);  
explicit bernoulli_distribution(double p);
```

Used on valarray: `explicit valarray(size_t);`.

Used on various templates in `linalg`, e.g.

```
// inside template<class Layout> class layout_transpose  
constexpr explicit mapping(const nested-mapping-type&);
```

§ 3.2.10 [time] (47 occurrences)

Used on various time templates, e.g.:

```
template<class Rep2>  
    constexpr explicit duration(const Rep2& r);  
constexpr explicit time_point(const duration& d);  
constexpr explicit day(unsigned d) noexcept;
```

`explicit` is also used on a few conversion operators in this clause, e.g.:

```
constexpr explicit day::operator unsigned() const noexcept;  
  
constexpr explicit month::operator unsigned() const noexcept;  
  
constexpr          year_month_day::operator sys_days()    const noexcept;  
constexpr explicit year_month_day::operator local_days() const noexcept;  
  
constexpr          year_month_day_last::operator sys_days()    const  
                    noexcept;  
constexpr explicit year_month_day_last::operator local_days() const  
                    noexcept;
```

The intention here appears to be to allow implicit conversions in some places, and to create other explicit conversions that require explicit opt-in, using a cast.

§ 3.2.11 [localization] (38 occurrences)

Used on various locales, facets, etc., e.g.:

```
explicit locale(const char* std_name);  
  
explicit collate_byname(const char*, size_t refs = 0);
```

§ 3.2.12 [input.output] (110 occurrences)

Used on various exception templates, e.g.:

```
explicit ios_base::failure(const string& msg, const error_code& ec =  
    io_errc::stream);
```

Used on various stream and buf templates, e.g.:

```
explicit basic_istream(basic_streambuf<charT, traits>* sb);
```

Used on various filesystem templates, e.g.:

```
explicit directory_iterator(const path& p);
```

Used in explicit operator bool() const noexcept.

§ 3.2.13 [re] (7 occurrences)

Used on an exception type:

```
explicit regex_error(regex_constants::error_type ecode);
```

Used on various regex templates, e.g.:

```
explicit basic_regex(const charT* p, flag_type f =  
    regex_constants::ECMAScript);  
explicit match_results(const Allocator& a);
```

§ 3.2.14 [thread] (43 occurrences)

Used on various templates related to threads, e.g.:

```
explicit stop_source(nostopstate_t) noexcept;  
  
template<class C>  
    explicit stop_callback(const stop_token& st, C&& cb)  
  
template<class F, class... Args> explicit thread(F&& f, Args&&... args);  
  
explicit atomic_ref(T&);
```

Used on various lock templates, e.g.:

```
explicit lock_guard(mutex_type& m);  
explicit scoped_lock(MutexTypes&... m);
```

Used on an exception type:

```
explicit future_error(future_errc e);
```

Used on various other templates, e.g.:

```
constexpr explicit counting_semaphore(ptrdiff_t desired);  
  
constexpr explicit latch(ptrdiff_t expected);  
  
constexpr explicit barrier(ptrdiff_t expected,  
    CompletionFunction f = CompletionFunction());
```

```
template<class F>
explicit packaged_task(F&& f);
```

§ 3.3 (3) A survey of the status quo for this topic, in the wider C++ community.

I did not look at this in any depth. Anecdotally, it seems very common to use `explicit` on constructors and `bool` conversion operators for disabling implicit conversions. I have rarely, if ever, seen `explicit` used to force the naming of a tag type at the point of construction outside of the standard.

§ 4 Proposal

§ 4.1 (4) A clear and concise definition of the policy proposed.

When proposing new templates and types for the standard library, `explicit` should be used to prevent unwanted implicit conversions, or to force the user to name a type at the point of its use when constructing values of that type. In particular, `explicit` should be applied in the following circumstances.

1. Place `explicit` on `bool` conversion operators, e.g. `constexpr explicit operator bool() const;`. Without this, the entire class type is convertible to `bool`. Example: `optional`'s `bool` conversion operator is `constexpr explicit operator bool() const noexcept` (see `[optional.observe]`). This makes `optional<T>` contextually convertible to `bool`, but it cannot directly be used to initialize a `bool`.
2. Place `explicit` on constructors that may take a single parameter (including when defaulted parameters are not passed), when implicit conversion from that single parameter is undesirable. Implicit conversions should exist only between types that are fundamentally the same (such as `float` and `double`, but not a `packaged_task` and some invocable that may be used to construct it). Example: `constexpr explicit vector(size_type n, const Allocator& = Allocator());` (see `[vector.cons]`).
3. Place `explicit` on tag types, to force the spelling of the tag type at the point of its construction. Example: `explicit unexpected_t() = default`, which prevents using `unexpected_t` by simply writing `{}` when calling a function that takes an `unexpected_t` at that argument position.

§ 4.2 (6) Proposed changes to the wording of SD-9.

Append to “List of Standard Library Policies” section of SD-9:

- X. (P3116) Place `explicit` on `bool` conversion operators.

X+1. (P3116) Place `explicit` on constructors that may be called with a single argument, when implicit conversion from that single argument is undesirable.

X+2. (P3116) Place `explicit` on nullary constructors of tag types.

§ 5 References

[P1976R2] Tomasz Kamiński. 2020-02-11. Fixed-size “span” construction from dynamic-size range.

<https://wg21.link/p1976r2>

[P2711R1] Ville Voutilainen. 2022-11-12. Making multi-param constructors of views explicit.

<https://wg21.link/p2711r1>